# 3GPP 6G THz Network Live Simulator

**Interactive Real-Life Use Case Simulation in Python**

This repository contains a high-fidelity, interactive, and live visual simulation of 3GPP 6G Terahertz (THz) networks. Built purely with Python and Matplotlib, it models complex real-world use cases using realistic physics, mobility models, and network parameters closely aligned with the **ns-3 network simulator** (specifically the TeraSim THz module).

## 🌟 Overview

The simulator visualizes various 6G environments ranging from 100 GHz to 1 THz. It calculates live radio frequency metrics—including Friis path loss, THz molecular absorption, penetration loss, and massive MIMO beamforming gain—to dynamically generate real-time Signal-to-Interference-Plus-Noise Ratio (SINR), throughput (Shannon capacity up to Tbps), and latency measurements.

## ✨ Key Features

- **Live Interactive UI:** Real-time dashboards displaying network topology, live SINR bars, throughput and latency timelines, SINR vs. Distance scatters, and CDFs.
- **Dynamic Heatmaps:** Pre-calculated, Gaussian-filtered THz propagation heatmaps for every environment.
- **NS-3 Physics Equivalent:** Implements physics mathematically aligned with ns3::ThzSpectrumPropagationLossModel and FriisPropagationLossModel.
- **Live Handover Tracking:** Monitors node transitions between gNBs/Access Points based on SINR thresholds.
- **Data Export:** Export live telemetry (X/Y coordinates, SINR, Throughput, Latency, Node Types) directly to CSV for post-processing.

## 🏭 Real-Life Use Cases Simulated

The simulation features 6 distinct environments, tailored with specific area dimensions, frequencies, bandwidths, and node mobility models:

1. **Holographic XR Surgery:** Operating room setting at 300 GHz prioritizing ultra-reliable low latency (URLLC < 1 ms) and 100 Gbps bandwidth for robot arms and holographic displays.

2. **Autonomous Factory:** Industry 6.0 smart factory at 140 GHz supporting URLLC and mMTC. Features roaming AGVs, static robot arms, and massive IoT sensor arrays.
3. **Smart City Intersection:** 6G V2X crossroad at 300 GHz. Focuses on zero-accident autonomous driving with strict <0.5 ms URLLC latency limits.
4. **6G Terabit Backhaul:** 1 THz point-to-point rooftop backhaul. Simulates massive eMBB (300 GHz bandwidth) achieving 1 Tbps per link with high-gain directional beamforming.
5. **Tunnel/Confined Rescue:** Confined space rescue ops at 100 GHz. Navigates high penetration losses with drones and body-worn radios for critical URLLC communication.
6. **Holographic Classroom:** Tele-education room at 300 GHz providing eMBB (10 Gbps/user) for 4K holographic displays per roaming student.

# ⚙ Requirements

This simulator is lightweight and relies solely on standard scientific Python libraries.

```bash
# Install required dependencies
pip install numpy scipy matplotlib
```

*Note: The script attempts to use the `TkAgg` backend for Matplotlib for optimal interactive performance. You may need to install `python-tk` depending on your OS (e.g., `sudo apt install python3-tk` on Ubuntu).*

# 🚀 Usage

Run the script directly from your terminal:

```bash
python 6g_thz_live_sim.py
```

## ⌨ Keyboard Controls

You can control the simulation using the on-screen clickable UI, or use the following keyboard shortcuts for rapid interaction:

| Key | Action |
|---|---|

| | |
|---|---|
| **SPACE** | Pause / Resume the simulation |
| **R** | Reset the current environment |
| **E** | Export current node telemetry to a CSV file |
| **1 – 6** | Quick-switch between the 6 different environments |
| **+ / =** | Increase simulation speed |
| **-** | Decrease simulation speed |
| **H** | Toggle THz Heatmap visibility |
| **T** | Toggle Node Trajectory Trails |
| **L** | Toggle gNB connection links |

## 📐 NS-3 Alignment Reference

For researchers looking to port these visual scenarios into actual ns-3 C++ scripts, the python logic aligns with the following models:

| **Simulation Logic** | **Equivalent C++ / NS-3 Module** |
|---|---|
| **Friis + THz Absorption PL** | ns3::ThzSpectrumPropagationLossModel |
| **Multi-beam MIMO BF** | ns3::ThzDirectionalAntenna |
| **Handover Trigger (SINR)** | ns3::LteHandoverAlgorithm |

| | |
|---|---|
| **Flow/Metrics Monitor** | ns3::FlowMonitor |
| **Traffic Generation** | ns3::UdpClientHelper |
| **Drone/Pedestrian Mobility** | ns3::RandomWalk2dMobilityModel |
| **AGV/Vehicle Mobility** | ns3::ConstantVelocityMobilityModel |
| **Static Sensors/Backhaul** | ns3::ConstantPositionMobilityModel |

# 🗁 Exporting Data

Clicking the 💾 **CSV** button (or pressing **E**) will snapshot the current state of the simulation and export it to your working directory.

**Format:** thz_export_[Environment_Name]_[Timestamp].csv

The CSV includes node IDs, categories, exact coordinates, live SINR (dB), Throughput (Gbps), Latency (ms), and the corresponding NS-3 mobility model.

## 1. Physics & Telemetry Calculations

These functions handle the core mathematical models aligned with the ns-3 network simulator:

- **thz_absorption_db(d_m, freq_hz)**: Calculates the molecular absorption loss specific to THz frequencies (100 GHz – 1 THz) over a given distance.
- **friis_db(d_m, freq_hz)**: Calculates the standard free-space path loss using the Friis transmission equation.
- **compute_sinr_thz(nx, ny, gnbs_cfg, env_cfg, rng)**: The master RF function. It calculates the live Signal-to-Interference-Plus-Noise Ratio (SINR) for a node based on its position, transmit power, beamforming gain, penetration loss, and the combined interference from all active gNBs (base stations).
- **shannon_tp(sinr_db, bw_ghz)**: Converts the calculated SINR into an estimated throughput (in Terabits per second) using the Shannon-Hartley theorem, scaled by a practical efficiency factor.

## 2. The Simulation Engine (SimState Class)

The SimState class acts as the brain of the simulation, keeping track of time, nodes, and history.

- **step(dt)**: This is the core engine function. It runs every frame to move the nodes based on their velocity/mobility models, recalculates their SINR/Throughput/Latency, logs the data into history arrays, and detects if a node has handed over to a new cell tower.
- **reset()**: Re-initializes all node positions, velocities, and telemetry histories when you switch environments or press the reset button.
- **export_csv()**: Takes a snapshot of the current node data and writes it out to a timestamped CSV file.

## 3. Graphics & Animation Loop

These functions draw the data onto the Matplotlib canvas:

- **build_heatmap_thz(env_cfg, res)**: A pre-computation function that calculates the underlying RF coverage heatmap for the selected environment using a grid and applies a Gaussian blur to smooth the visuals.
- **sinr_qcolor(v)**: A simple helper function that assigns a color (Green, Orange, Red) based on how strong the SINR value is.
- **draw_frame(_fn)**: **This is the most active function in the script.** Driven by Matplotlib's FuncAnimation, it triggers SIM.step() to advance the math, clears all the subplots, and completely redraws the topology map, live SINR bars, timeline graphs, scatter plots, and CDFs.

## 4. Interactive UI Callbacks

These are event-listener functions that trigger when you click buttons or use your keyboard:

- **on_env(label)**: Triggers when you select a different Use Case (like "Auto Factory" or "Holo Classroom") and tells the system to switch configs and reset.
- **on_svc(label) / on_disp(label)**: Triggers when you toggle checkboxes to hide/show specific services (URLLC, eMBB) or visual elements (Heatmaps, Trails).
- **on_pause(ev)**: Pauses or resumes the SIM.step() progression.
- **on_key(ev)**: Listens for keyboard shortcuts (like SPACE to pause, 1-6 to switch environments, or + / - to change the simulation speed).